

JAVA BACKEND OS SERIES

운영체제 핵심 정리

STEP 1 · Kernel, Process, Thread & Scheduling

중심 장애 가상 장비 연결마다 플랫폼 스레드를 무제한 생성해 스택·네이티브 메모리·스케줄링 자원이 고갈되는 문제

Java 21 이상 · com.factorypulse · FactoryPulse TCP 수집 서버

선수 단계: Step 0, '전체 학습 지도·선수관계·진단'

CONTENTS

목차와 학습 목표

30쪽 구성

장	내용	쪽
1	선수지식·핵심 경계	3-5
2	프로그램·프로세스·스레드	6-11
3	문맥 교환·CPU 스케줄링	12-14
4	무제한 스레드 장애·코드·테스트	15-20
5	실무 확인·면접·과제	21-24
부록	문제·해설·체크리스트·공식 자료	25-30

학습 목표

- 사용자 모드와 커널 모드의 책임 경계를 시스템 콜 시간축으로 설명한다.
- 프로그램·프로세스·스레드의 자원 소유 관계를 FactoryPulse JVM에 매핑한다.
- Java Thread.State와 OS 스케줄 상태가 동일하지 않음을 설명한다.
- 무제한 플랫폼 스레드 생성의 자원 비용을 재현하고 상한이 있는 구조로 수정한다.
- 장애 증상에서 thread dump·OS thread 수·문맥 교환 지표를 선택한다.

01 · PREREQUISITE

Step 0 연결 요약과 선수지식 진단

이 페이지를 넘는 복습은 Step 0, 4-10-13절을 참조

1페이지 요약 프로세스는 OS 자원을 가진 실행 인스턴스이고, 스레드는 그 안의 실행 흐름이다. 플랫폼 스레드는 일반적으로 OS 커널 스레드에 1:1로 매핑되어 제한된 자원이다. [R1]

진단 1. Java 메서드 호출과 시스템 콜은 같은가? 차이를 한 문장으로 쓰라.

답안

답안

진단 2. 프로세스의 모든 스레드가 공유하는 것과 공유하지 않는 것을 각각 하나 쓰라.

답안

답안

진단 3. Thread.State.RUNNABLE이면 반드시 CPU에서 실행 중인가?

답안

답안

진단 4. 스레드 10,000개가 CPU를 거의 쓰지 않아도 위험할 수 있는 이유는?

답안

답안

02 · PRIVILEGE

사용자 모드와 커널 모드

핵심 정의: 하드웨어와 커널 자료구조를 보호하기 위한 CPU 권한 수준의 분리

Java 코드에서 장치까지: 권한 경계를 왕복



그림 1. Java 애플리케이션은 JDK/JVM을 통해 커널 서비스에 도달한다.

구분	사용자 모드	커널 모드
주체	애플리케이션-대부분의 JVM 코드	커널-드라이버
권한	제한된 메모리/명령	특권 명령-커널 주소 공간
실패 영향	보통 해당 프로세스 격리	버그가 시스템 전체에 영향 가능
FactoryPulse	이벤트 파싱-업무 로직	socket read, 스케줄링, NIC 처리

주의 'JVM은 커널 모드에서 실행된다'는 표현은 틀리다. JVM 프로세스의 대부분 코드는 사용자 모드이며 필요한 순간 시스템 콜로 커널에 진입한다.

02 · PRIVILEGE

시스템 콜의 실제 시간축

시스템 콜은 메서드 호출보다 비싸다는 단점 대신 경계와 대기를 구분한다

순서	실행 주체	핵심 동작
1	Java/JDK	Socket 입력 API 호출, 인자-버퍼 준비
2	JVM/native library	운영체제 API에 맞는 호출로 연결
3	CPU/커널	권한 전환, 인자-FD 검증, 소켓 상태 확인
4	커널	데이터가 없으면 스레드를 대기 상태로 두고 다른 작업 스케줄
5	NIC/커널	패킷 도착 후 대기 작업을 runnable로 전환
6	JVM/Java	결과 반환, 파싱-저장 계속

면접 포인트 시스템 콜 진입 자체와 스레드 문맥 교환은 같은 사건이 아니다. 즉시 완료되는 호출은 다른 스레드로 전환하지 않을 수 있고, blocking-선점-깨움 조건에서 전환이 발생할 수 있다.

Linux의 정확한 ABI, 호출 번호, vDSO 사용 여부는 아키텍처와 커널-libc 구현에 따라 달라진다. [R5]

03 · EXECUTION UNIT

Program과 Process

파일에 저장된 코드가 실행되면 OS가 자원 컨테이너를 만든다

개념	한 문장 정의	FactoryPulse 예
Program	실행 가능한 코드와 정적 데이터	factorypulse.jar
Process	실행 중인 프로그램 인스턴스와 OS 자원 묶음	java -jar 실행으로 생긴 JVM PID
PID	OS가 프로세스를 식별하는 값	관측 명령·로그의 pid
주소 공간	프로세스가 보는 가상 주소 범위	heap, thread stacks, native memory

프로세스가 소유하거나 참조하는 대표 자원

- 가상 주소 공간과 메모리 매핑 - 상세는 Step 2, '가상 메모리·페이지'
- 열린 파일·소켓을 나타내는 FD - 상세는 Step 5, 'File Descriptor·Socket'
- 보안 자격, 작업 디렉터리, 시그널 처리 상태
- 하나 이상의 실행 스레드와 스케줄링·사용량 통계

구현 차이 Linux에서는 커널 내부의 task 개념과 thread group을 통해 프로세스/스레드를 표현한다. Windows의 객체 모델과 용어는 다르다.

03 · EXECUTION UNIT

프로세스 내부 동작과 생성

OS가 '프로세스 전체'를 매 순간 CPU에 올리는 것이 아니다

한 JVM 프로세스 안의 여러 플랫폼 스레드

JVM 프로세스	Thread A	Thread B	공유
● PID-주소 공간 ● 열린 소켓/파일 ● 힙 코드-메타데이터	● 스택-PC-레지스터 ● 장비 001 연결 ● OS 스케줄 대상	● 스택-PC-레지스터 ● 장비 002 연결 ● OS 스케줄 대상	● Java heap ● 클래스-FD 테이블 ● 프로세스 한도

그림 2. JVM 프로세스의 자원 공유와 스레드별 실행 문맥.

공유	스레드별
Java heap, class metadata, code cache	Java/native stack
열린 파일-socket 테이블에 대한 프로세스 참조	program counter-레지스터 문맥
프로세스 주소 공간과 라이브러리 매핑	스케줄 상태-CPU 사용 통계

주의 '프로세스 문맥 교환'이라는 말은 편의상 쓰지만, 현대 OS 스케줄러가 실제로 선택하는 단위는 보통 thread/task이다. 주소 공간까지 바뀌는 전환은 같은 프로세스 내 스레드 전환보다 추가 비용이 생길 수 있다.

03 · EXECUTION UNIT

Thread의 자원과 생명주기

실행 순서·스택·레지스터 문맥을 가지며 OS 스케줄링의 대상이 된다

단계	Java 관점	OS 관점
생성	Thread 객체·Runnable 준비	start 전에는 실행 thread가 아직 없을 수 있음
시작	start()가 run() 실행을 예약	플랫폼 스레드/커널 task 생성·runnable
실행	run()의 Java 코드 수행	CPU에 배정되거나 run queue 대기
대기	sleep/read/lock 등	원인에 따라 sleep/blocked/runnable
종료	run() 정상/비정상 완료	스케줄 대상 제거·자원 정리

핵심 Thread 객체를 new 했다고 별도 실행 흐름이 시작되는 것은 아니다. start()가 새 실행을 예약하며 run()을 직접 호출하면 현재 스레드에서 평범한 메서드처럼 실행된다. [R1]

장애 연결 연결당 플랫폼 스레드는 각 장비가 느리거나 조용해도 스택과 OS/JVM 관리 자원을 점유한다.

Runnable과 Thread 최소 예제

작업(Runnable)과 실행 주체(Thread)를 분리한다

```
package com.factorypulse.step1;

import java.util.concurrent.CountDownLatch;

public final class ThreadStateDemo {
    public static void main(String[] args) throws Exception {
        var entered = new CountDownLatch(1);
        Thread worker = Thread.ofPlatform().name("equipment-reader").unstarted(() -> {
            entered.countDown();
            try {
                Thread.sleep(300);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });

        System.out.println("before=" + worker.getState());
        worker.start();
        entered.await();
        Thread.sleep(30);
        System.out.println("during=" + worker.getState());
        worker.join();
        System.out.println("after=" + worker.getState());
    }
}
```

예상 실행 before=NEW
during=TIMED_WAITING
after=TERMINATED

- Thread.ofPlatform()은 Java 21의 플랫폼 스레드 builder이다.
- name()은 thread dump와 로그에서 장비 연결을 식별하게 한다.
- join()은 대상 스레드 종료를 기다린다.
- sleep()은 TIMED_WAITING을 만들지만 정확한 재개 시각을 보장하지 않는다.

Java Thread.State 읽는 법

Java 상태는 VM 상태이며 OS 상태와 일대일로 대응하지 않는다. [R2]

Java Thread.State는 OS 상태표가 아니다

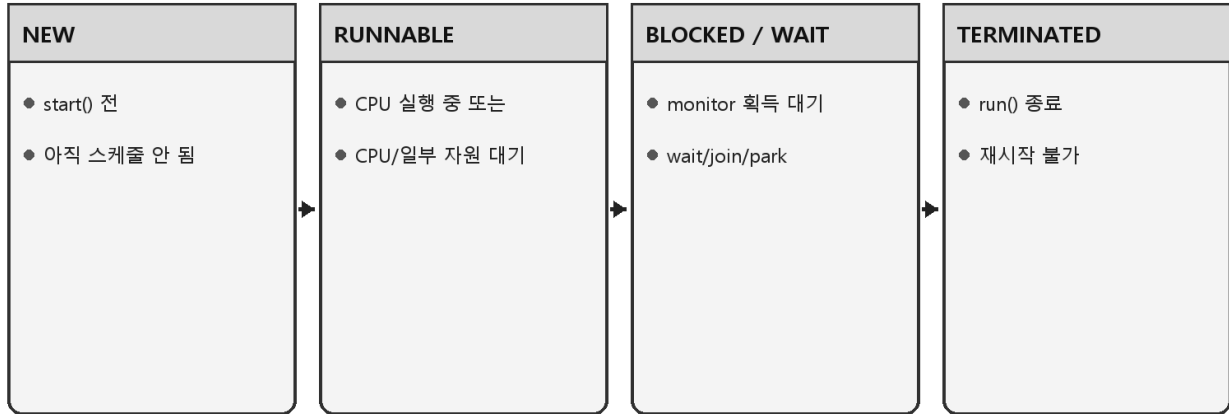


그림 3. 주요 Java 상태 전이. 실제 전이는 호출과 동기화 조건에 따라 달라진다.

상태	뜻	대표 원인
NEW	아직 시작하지 않음	unstarted() / new Thread
RUNNABLE	JVM에서 실행 가능/실행 중	CPU 실행, CPU 대기, 일부 native I/O
BLOCKED	monitor lock 획득 대기	synchronized 진입
WAITING	기한 없는 특정 동작 대기	wait(), join(), park()
TIMED_WAITING	기한 있는 대기	sleep(), timed join
TERMINATED	실행 완료	run() 반환 또는 예외 종료

주의 RUNNABLE thread 수가 많다고 모두 CPU를 동시에 쓰는 것은 아니다. 코어 수보다 많은 runnable 작업은 스케줄 대기과 전환을 만든다.

Platform Thread의 비용과 한계

Java 21 API는 플랫폼 스레드를 보통 OS 커널 스레드에 1:1로 매핑되는 제한 자원으로 설명한다. [R1]

자원	증가 원인	관찰 증상
스택/네이티브 메모리	플랫폼 스레드마다 stack 예약-commit	RSS/virtual size 증가, thread 생성 실패
커널 관리 구조	OS thread/task 증가	PID/TID 수와 한도 압박
스케줄링 비용	runnable thread 증가	context switch, run queue, p99 증가
캐시/TLB 지역성	자주 실행 대상 변경	CPU는 높지만 유효 처리량 감소 가능

조건부 정확한 stack 크기와 commit 방식은 JVM 옵션(-Xss), JDK/JVM 구현, OS, 가상 주소 공간 정책에 따라 다르다. '스레드 1개=정확히 1MB 물리 메모리'처럼 고정값으로 말하지 않는다.

Step 4 참조 가상 스레드는 Step 4에서 본격 비교한다. Step 1에서는 플랫폼 스레드가 제한 자원이라는 기준만 확립한다. [R3]

05 · CONTEXT SWITCH

문맥 교환: 무엇을 저장하고 왜 비용이 드나

CPU가 실행 대상을 바꾸기 위해 이전 실행 문맥을 보존하고 다음 문맥을 복원하는 과정

문맥 교환 시간축

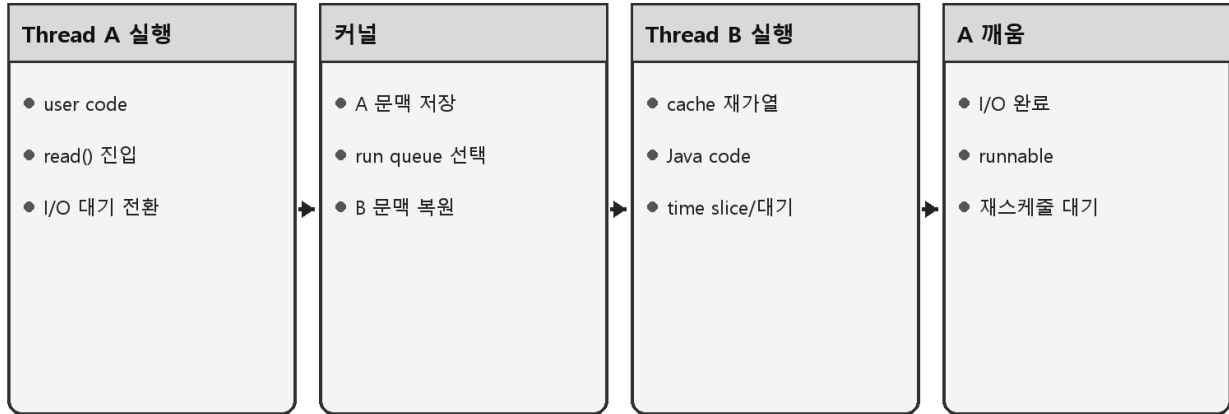


그림 4. blocking I/O 뒤 다른 스레드가 실행되고, 완료 후 원래 스레드가 다시 경쟁한다.

직접 비용	간접 비용
레지스터·스택 포인터·프로그램 카운터 저장/복원	CPU cache·TLB·branch predictor 지역성 약화
스케줄러 자료구조 갱신·대상 선택	run queue가 길 때 응답 지연 분산 확대

오해 문맥 교환 횟수가 0이어야 좋은 것이 아니다. I/O 서버에는 정상적인 대기·깨움이 필요하다. 처리량/지연과 함께 과도한지 판단한다.

05 · SCHEDULING

CPU Scheduling

runnable 작업 중 어떤 스레드를 언제 어느 CPU에 실행할지 결정한다

개념	의미	FactoryPulse 영향
run queue	실행 가능한 task 대기 구조	파싱·압축 작업 폭주 시 길어짐
preemption	실행 중 task의 CPU를 회수	지연 민감 작업 응답성 확보
priority/nice	CPU 배분에 영향을 주는 정책 입력	Java priority와 OS 반영은 플랫폼 의존
CPU affinity	실행 가능한 CPU 집합 제한	cache locality와 불균형 모두 가능
load balancing	CPU 간 runnable 작업 이동	멀티코어 활용·이동 비용

Linux 구현 차이 Linux의 일반 공정 스케줄링 구현은 커널 버전에 따라 진화한다. CFS의 vruntime 설명만 현재 모든 커널의 선택 규칙으로 단정하지 않는다. Linux 6.6부터 EEVDF로의 전환이 시작됐다. [R6][R7]

면접 포인트 CPU-bound 작업은 코어 수를 크게 넘는 thread가 처리량을 늘리지 못하고 전환 비용을 키울 수 있다. I/O-bound 작업은 대기 때문에 더 많은 동시성이 유효할 수 있지만 자원 상한은 여전히 필요하다.

05 · END-TO-END

검사 이벤트 한 건의 실행 시간축

OS → JVM → 애플리케이션 경계를 한 요청에 연결

시각	계층	사건	스레드 상태/관찰
t0	커널	NIC가 TCP 데이터 수신	socket receive queue
t1	커널	대기 thread 깨움	sleep→runnable
t2	스케줄러	equipment-worker 배정	run queue→CPU
t3	JVM/Java	read 반환·프레임 파싱	RUNNABLE
t4	Java/JDBC	DB 저장 요청	향후 Step 7: pool wait
t5	커널	ACK/write 처리	system call-buffer
t6	스케줄러	다른 장비 thread 실행	context switch 가능

장애 단서 장비가 10,000대이고 각 연결 thread가 대부분 read 대기라면 CPU가 낮아도 thread 수·스택·FD가 먼저 한계에 도달할 수 있다. 반대로 이벤트가 동시에 몰리면 runnable 수와 CPU 경쟁이 급증한다.

TCP 스트림·부분 읽기·timeout·FD 누수는 Step 5에서 상세히 다룬다.

06 · FAILURE CASE

장애 사례: 연결당 Thread 무제한 생성

정상 시 단순하지만 연결 수가 곧 플랫폼 스레드 수가 된다

연결당 플랫폼 스레드 무제한 생성의 장애 연쇄

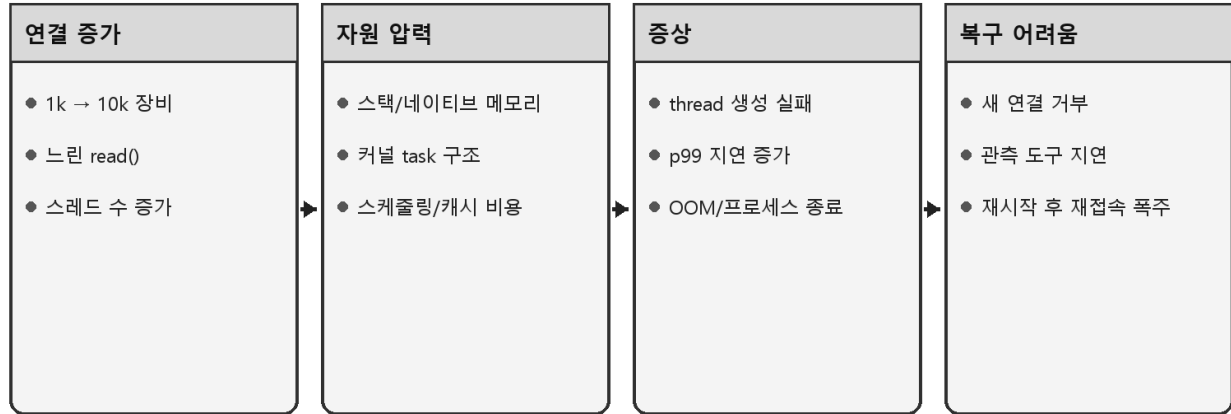


그림 5. 연결 증가가 스택:커널:스케줄러 압력과 복구 실패로 연쇄된다.

발생 조건

- 장비 연결 수에 상한이 없고 연결마다 Thread.ofPlatform().start() 호출
- 장비가 오래 연결되거나 read에서 장시간 대기
- 재연결 폭주로 짧은 시간에 thread 생성 집중
- -Xss, pid/thread limit, 컨테이너 메모리 한계와 결합

대표 로그 java.lang.OutOfMemoryError: unable to create native thread — heap 부족과 다른 경로일 수 있으므로 OS thread 한도와 native memory를 함께 확인한다.

06 · FAILURE CASE

잘못된 코드

컴파일은 되지만 연결 수를 자원 상한으로 직접 전파한다

```
package com.factorypulse.step1;

import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.net.ServerSocket;
import java.net.Socket;
import java.nio.charset.StandardCharsets;

public final class UnboundedEquipmentServer {
    public static void main(String[] args) throws IOException {
        try (var server = new ServerSocket(9000)) {
            while (true) {
                Socket socket = server.accept();
                Thread.ofPlatform().name("equipment-" + socket.getPort()).start(() -> handle(socket));
            }
        }

        private static void handle(Socket socket) {
            try (socket;
                var reader = new BufferedReader(new InputStreamReader(
                    socket.getInputStream(), StandardCharsets.UTF_8))) {
                while (reader.readLine() != null) {
                    // A slow device keeps this platform thread occupied.
                }
            } catch (IOException ignored) {
                // Logging is omitted to keep the example minimal.
            }
        }
    }
}
```

문제점 accept 성공 때마다 새 플랫폼 스레드를 시작한다. 장비가 데이터를 보내지 않아도 handle()의 readLine()이 끝날 때까지 스레드가 유지된다.

추가 주의 readLine()은 예시 단순화를 위한 것이며 실제 바이너리 검사 프레임에는 부적합하다. framing은 Step 5 참조.

06 · FAILURE CASE

안전한 장애 재현과 관찰

운영 서버에서 무제한 실험을 하지 말고 상한이 있는 시뮬레이터로 추세를 본다

```
package com.factorypulse.step1;

import java.util.ArrayList;
import java.util.concurrent.CountDownLatch;

public final class ThreadGrowthSimulator {
    public static void main(String[] args) throws Exception {
        int limit = args.length == 0 ? 200 : Math.min(Integer.parseInt(args[0]), 1_000);
        var release = new CountDownLatch(1);
        var threads = new ArrayList<Thread>();
        for (int i = 1; i <= limit; i++) {
            Thread thread = Thread.ofPlatform().name("blocked-device-" + i).start(() -> await(release));
            threads.add(thread);
            if (i % 50 == 0) System.out.println("created=" + i);
        }
        System.out.println("activePlatformThreads=" + threads.size());
        release.countDown();
        for (Thread thread : threads) thread.join();
    }

    private static void await(CountDownLatch latch) {
        try {
            latch.await();
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}
```

실행 `java com.factorypulse.step1.ThreadGrowthSimulator 200`

예상 출력 created=50
created=100
created=150
created=200
activePlatformThreads=200

- 작업 관리자/ps에서 thread 수 증가를 확인한다.
- 더 큰 수는 격리된 컨테이너에서만 단계적으로 시도한다.
- 실험 전 메모리·PID 한도와 종료 경로를 확인한다.

07 · REMEDIATION

수정 구조: 수락량과 실행량에 상한을 둔다

연결 수·worker 수·queue 길이·timeout은 별도 정책이다

경계	정책 예	효과
WORKERS	32개 플랫폼 스레드 고정	동시 실행/stack 상한
QUEUE_CAPACITY	128개 대기 작업	대기 메모리 상한
포화 정책	새 연결 즉시 종료 또는 프로토콜 수준 거부	무제한 누적 차단
read timeout	Step 5에서 설정	느린/유휴 연결 회수
관측	active, queue, reject, latency	포화 전 경보

주의 고정 thread pool은 연결을 32개만 유지하라는 뜻이 아니다. blocking 연결당 작업 모델에서는 worker가 연결 수를 제한한다. 많은 장기 연결에는 NIO/Selector 또는 Java 21 가상 스레드 같은 다른 실행 모델을 Step 4·5에서 평가한다.

핵심 이번 Step의 수정 목표는 ‘무제한’을 제거하는 것이다. 최적 숫자는 장비 수, blocking 시간, CPU, 메모리, DB 처리량을 측정해 정한다.

07 · REMEDIATION

수정 코드

bounded worker + bounded queue + 명시적 거부

```
package com.factorypulse.step1;

import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.net.ServerSocket;
import java.net.Socket;
import java.nio.charset.StandardCharsets;
import java.util.concurrent.ArrayBlockingQueue;
import java.util.concurrent.ThreadPoolExecutor;
import java.util.concurrent.TimeUnit;

public final class BoundedEquipmentServer {
    private static final int WORKERS = 32;
    private static final int QUEUE_CAPACITY = 128;

    public static void main(String[] args) throws IOException {
        var factory = Thread.ofPlatform().name("equipment-worker-", 0).factory();
        try (var workers = new ThreadPoolExecutor(
            WORKERS, WORKERS, 0L, TimeUnit.MILLISECONDS,
            new ArrayBlockingQueue<>(QUEUE_CAPACITY), factory,
            new ThreadPoolExecutor.AbortPolicy());
            var server = new ServerSocket(9000)) {
            while (!workers.isShutdown()) {
                Socket socket = server.accept();
                try {
                    workers.execute(() -> handle(socket));
                } catch (RuntimeException rejected) {
                    socket.close(); // Close explicitly on saturation.
                }
            }
        }

        private static void handle(Socket socket) {
            try (socket;
                var reader = new BufferedReader(new InputStreamReader(
                    socket.getInputStream(), StandardCharsets.UTF_8))) {
                reader.readLine(); // Step 5 adds framing and timeouts.
            } catch (IOException ignored) {}
        }
    }
}
```

자원 특성 platform thread 최대 32, 대기 Runnable 최대 128. queue 포화 후에는 작업을 무한 대기시키지 않고 소켓을 닫는다.

Step 4 참조 ThreadPoolExecutor의 sizing-거부 정책-backpressure 상세는 Step 4, 'Thread Pool-Queue-포화 정책'에서 설명한다.

08 · TEST

검증 테스트

두 worker와 두 queue 슬롯이 모두 찬 뒤 다섯 번째 작업이 거부되는지 확인

```
package com.factorypulse.step1;

import java.util.concurrent.ArrayBlockingQueue;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.RejectedExecutionException;
import java.util.concurrent.ThreadPoolExecutor;
import java.util.concurrent.TimeUnit;

public final class BoundedExecutorTest {
    public static void main(String[] args) throws Exception {
        var blocker = new CountDownLatch(1);
        var executor = new ThreadPoolExecutor(
            2, 2, 0L, TimeUnit.MILLISECONDS,
            new ArrayBlockingQueue<>(2),
            Thread.ofPlatform().name("test-worker-", 0).factory(),
            new ThreadPoolExecutor.AbortPolicy());

        try {
            Runnable waitingTask = () -> await(blocker);
            for (int i = 0; i < 4; i++) executor.execute(waitingTask);

            boolean rejected = false;
            try {
                executor.execute(() -> { });
            } catch (RejectedExecutionException expected) {
                rejected = true;
            }
            if (!rejected || executor.getPoolSize() > 2 || executor.getQueue().size() > 2) {
                throw new AssertionError("resource bound violated");
            }
            System.out.println("PASS workers<=2 queue<=2 fifth=rejected");
        } finally {
            blocker.countDown();
            executor.shutdown();
            if (!executor.awaitTermination(2, TimeUnit.SECONDS)) executor.shutdownNow();
        }
    }

    private static void await(CountDownLatch latch) {
        try {
            latch.await();
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}
```

실행 결과 PASS workers<=2 queue<=2 fifth=rejected

검증	실패 시 의미
poolSize ≤ 2	worker 상한이 깨짐
queueSize ≤ 2	대기 작업 메모리 상한이 깨짐
5번째 rejected	포화가 호출자에게 전달되지 않음
종료 2초 내 완료	interrupt/정리 경로 문제

테스트는 네트워크 자체보다 자원 상한 계약을 검증한다. socket framing과 느린 client 통합 테스트는 Step 5에서 추가한다.

Thread·메모리·CPU·Queue 자원표

숫자는 예제가 아니라 부하시험으로 확정

자원	무제한 코드	수정 코드	주요 지표
Thread	연결 수와 함께 증가	최대 32	live/peak thread, create failure
Stack/native	thread 수만큼 압력	상한 예측 가능	RSS, NMT, -Xss
CPU	동시 이벤트 때 runnable 폭주	실행 동시성 제한	CPU, run queue, context switch
Queue	암묵적으로 연결/thread에 누적	128 작업	queue size, reject count
Socket	종료까지 유지	거부 시 즉시 close	open FD, connection count

운영체제 차이 Linux는 /proc, ps, pidstat, perf를 활용할 수 있다. Windows는 Process Explorer/성능 카운터/ETW 등 관측 경로가 다르다. Java Thread.State는 두 OS 모두 OS 상태를 그대로 노출하지 않는다.

JVM 차이 플랫폼 스레드와 native stack의 실제 메모리 사용량·생성 한계는 HotSpot/OpenJ9, OS, -Xss, ulimit/cgroup 등에 따라 달라진다.

10 · PRACTICE

실무 확인 방법

증상 하나를 단일 지표로 확정하지 않는다

질문	Java/JDK	Linux 예
thread가 몇 개인가?	jcmd <pid> Thread.print; JFR	ps -L -p <pid>; /proc/<pid>/task
어디서 대기하나?	thread dump의 state/stack	pidstat -w; perf sched
native memory 압력?	jcmd <pid> VM.native_memory summary	pmap -x; /proc/<pid>/status
스케줄 경쟁?	JFR CPU/thread events	vmstat; pidstat -w; /proc/loadavg
한도는?	-Xss, JVM flags	ulimit -u/-s; cgroup pids.max/memory.max

실무 확인 순서 1) live thread-FD-RSS 추세 → 2) thread dump 상태 분포 → 3) run queue-context switch → 4) stack/queue 원인 → 5) 제한값과 배포 환경 비교

NMT는 시작 시 옵션이 필요할 수 있고 모든 native allocation을 완전하게 설명하지 않는다. 명령 지원과 출력은 JDK/OS 버전을 확인한다. [R8]

11 · INTERVIEW

자주 하는 오해와 면접 답변

정의 → 내부 동작 → 장애 → 확인 → 대안 순서

오해/위험한 답변	좋은 답변
시스템 콜이면 무조건 문맥 교환한다.	권한 전환과 스레드 전환을 구분하고 blocking-선점 조건을 말한다.
RUNNABLE은 CPU 실행 중이다.	Java RUNNABLE에는 CPU 대기 등도 포함되며 OS 상태와 1:1이 아니라고 말한다.
CPU가 낮으니 thread는 많아도 된다.	stack/native memory-PID 한도·깨움 폭주·운영성 비용을 함께 본다.
thread pool 200개가 정답이다.	작업의 CPU/I/O 비율, downstream 처리량, 메모리, queue 정책으로 검증한다.
virtual thread면 모든 문제가 끝난다.	동시 task는 늘릴 수 있어도 DB/FD/메모리/backpressure 한계는 남는다고 말한다.

모범 90초 답변 플랫폼 스레드는 일반적으로 OS 커널 스레드에 1:1로 매핑되는 제한 자원입니다. 연결당 무제한 생성하면 느린 장비가 CPU를 거의 쓰지 않아도 stack과 native-kernel 자원을 점유합니다. 이벤트가 몰리면 runnable 증가와 문맥 교환으로 p99도 악화됩니다. 우선 live thread, 상태 분포, RSS, FD, context switch를 함께 확인하고, worker-queue-timeout에 상한을 둔 뒤 NIO나 가상 스레드는 부하 특성에 맞춰 평가하겠습니다.

12 · ASSIGNMENT

직접 구현 과제

정답 코드보다 측정 증거가 산출물

과제	필수 조건	통과 기준
A. 상태 관찰기	NEW→TIMED_WAITING→TERMINATED 출력	3회 반복해 상태와 이유 설명
B. thread 성장 실험	50/100/200 단계, 종료 latch 포함	thread·RSS 추세 표 작성
C. bounded 수집기	worker/queue/reject 설정 외부화	상한 테스트와 reject metric
D. 장애 보고서	증상·가설·증거·수정·회귀 테스트	언어/JVM/OS 층위 구분

구현 제한

- 운영 장비망에서 thread exhaustion 실험 금지
- 소켓은 try-with-resources 또는 명시적 close
- InterruptedException을 삼키지 말고 interrupt 상태 복원
- 상한값은 하드코딩 결과가 아니라 부하시험 근거와 함께 제출

통과 질문 '왜 worker 32인가?'에 CPU 코어 수 하나만 답하지 말고 blocking 비율, stack/native memory, DB 처리량, queue wait, p99 근거를 제시한다.

13 · REVIEW

복습 문제

문항 간격과 답안선 간격을 넓게 배치

1. 사용자 모드와 커널 모드를 나누는 목적을 보안과 안정성 관점에서 설명하라.

답안

답안

2. 시스템 콜과 문맥 교환이 같은 개념이 아닌 이유를 쓰라.

답안

답안

3. 프로세스 자원과 스레드별 자원을 각각 두 가지 쓰라.

답안

답안

4. Java RUNNABLE과 OS running을 동일시하면 안 되는 이유는?

답안

답안

5. 연결당 플랫폼 스레드 모델에서 CPU가 낮아도 장애가 날 수 있는 경로를 쓰라.

답안

답안

APPENDIX A

복습 해설과 화이트보드 5문제

먼저 25쪽을 푼 뒤 확인

#	핵심 해설
1	특권 자원 접근을 커널로 제한해 프로세스 결함을 격리한다.
2	시스템 콜은 권한 경계 진입, 문맥 교환은 실행 스레드 변경이다. 전자는 후자를 항상 요구하지 않는다.
3	공유: 주소 공간·heap·FD. 스레드별: stack·PC/레지스터·스케줄 상태.
4	Thread.State는 VM 상태이며 RUNNABLE에 CPU 대기 등이 포함될 수 있다.
5	stack/native-kernel task·PID/FD 한도, 깨움 폭주, context switch로 고갈/지연이 가능하다.

화이트보드 문제

- WB1. 1 JVM·4코어·2,000 연결을 프로세스/스레드/코어 그림으로 표현하라.
- WB2. read blocking부터 다른 thread 실행까지 시간축을 그려라.
- WB3. Java 상태와 OS 상태를 두 층으로 그려라.
- WB4. 연결당 thread 구조의 자원 증가 그래프를 그려라.
- WB5. worker-queue-reject 경계를 포함한 수정 구조를 그려라.

APPENDIX B

코드 리뷰 3문제·장애 분석 3문제

각 문제 아래 근거와 확인 지표를 함께 적는다

CR1. `new Thread(task).run()`을 사용한 코드를 리뷰하라.

답안

CR2. `accept`마다 `Thread.ofPlatform().start()`를 호출하는 코드를 리뷰하라.

답안

CR3. `InterruptedException`을 `catch`한 뒤 아무것도 하지 않는 코드를 리뷰하라.

답안

INC1. CPU 25%, live thread 8,000, RSS 급증: 우선 가설과 확인 순서를 쓰라.

답안

INC2. CPU 100%, runnable 600, 처리량 정체: 스케줄링 관점 가설을 쓰라.

답안

INC3. 재시작 직후 thread가 급증: 장비 재접속 폭주를 완화할 경계를 쓰라.

답안

APPENDIX C

기술면접 질문 10개

각 60~120초 답변

I1. Program과 Process의 차이는?

답안

I2. Process와 Thread는 무엇을 공유하는가?

답안

I3. User mode와 Kernel mode를 왜 나누는가?

답안

I4. System call과 일반 메서드 호출의 차이는?

답안

I5. Context switch 비용의 직접·간접 요소는?

답안

I6. Java Thread.State와 OS 상태의 관계는?

답안

I7. Platform thread가 제한 자원인 이유는?

답안

I8. CPU-bound와 I/O-bound의 thread sizing 차이는?

답안

I9. 연결당 thread 장애를 어떻게 확인하는가?

답안

I10. bounded queue가 왜 필요한가?

답안

APPENDIX D

30분 복습 체크리스트와 공식 자료

면접 직전: 0-10분 정의, 10-20분 그림, 20-30분 장애 답변

분	확인
0-5	mode/system call/process/thread 정의를 소리 내어 설명
5-10	Thread.State 6개와 RUNNABLE 함정
10-15	시스템 콜·blocking·문맥 교환 시간축 그리기
15-20	JVM process 안의 공유/스레드별 자원 그리기
20-25	무제한 thread 장애의 지표·복구 설명
25-30	90초 모범 답변과 꼬리 질문 3개

공식 자료

- [R1] Oracle Java SE 21 API, java.lang.Thread — <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/Thread.html>
- [R2] Oracle Java SE 21 API, Thread.State — <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/Thread.State.html>
- [R3] OpenJDK JEP 444, Virtual Threads — <https://openjdk.org/jeps/444>
- [R4] JLS SE 21, Chapter 17 — <https://docs.oracle.com/javase/specs/jls/se21/html/jls-17.html>
- [R5] Linux man-pages: clone(2), sched(7), getrusage(2) — <https://man7.org/linux/man-pages/>
- [R6] Linux Kernel Docs, EEVDF — <https://docs.kernel.org/scheduler/sched-eevdf.html>
- [R7] Linux Kernel Docs, CFS — <https://docs.kernel.org/scheduler/sched-design-CFS.html>
- [R8] Oracle JDK 21 Troubleshooting/Tools — <https://docs.oracle.com/en/java/javase/21/troubleshoot/>

14 · ONE-PAGE SUMMARY

한 페이지 요약

Step 2로 자동 진행하지 않음

질문	핵심 답
권한 경계?	Java/JVM 대부분은 user mode; 커널 서비스는 system call로 요청.
실행 단위?	프로세스는 자원 컨테이너, 스레드는 스택·문맥을 가진 스케줄 단위.
Java 상태?	VM 상태이며 OS 상태와 1:1이 아니다. RUNNABLE=항상 CPU 실행이 아님.
문맥 교환?	실행 대상 변경. 모드 전환과 구분하고 cache/TLB 간접 비용도 본다.
스케줄링?	runnable task를 CPU에 배정. 정책·구현은 OS/커널 버전에 의존.
중심 장애?	연결당 플랫폼 스레드 무제한 생성→stack/native/kernel/scheduler 압력.
수정 원칙?	worker-queue-timeout-accept에 상한, 포화 정책과 지표를 명시.
확인 증거?	live/state thread, RSS/native, FD, run queue, context switch, reject.

면접 한 문장 플랫폼 스레드는 제한된 OS/JVM 자원이므로 연결 수를 그대로 스레드 수로 만들지 않고, 실행·대기·거부의 상한을 관측 가능한 정책으로 설계한다.

Step 1 통과 기준

- 시스템 콜과 문맥 교환을 구분해 설명
- Java 상태와 OS 상태를 구분
- 잘못된 서버·수정 서버·상한 테스트의 의도 설명
- FactoryPulse 장애를 90초 안에 지표와 함께 답변